
Lecture Notes for Chapter 16:

Greedy Algorithms

Chapter 16 Introduction

Similar to dynamic programming.
Used for optimization problems.

Idea

When we have a choice to make, make the one that looks best *right now*. Make a *locally optimal choice* in hope of getting a *globally optimal solution*.

Greedy algorithms don't always yield an optimal solution. But sometimes they do. We'll see a problem for which they do. Then we'll look at some general characteristics of when greedy algorithms give optimal solutions.

[We do not cover Huffman codes or matroids in these notes.]

Activity selection

n **activities** require *exclusive* use of a common resource. For example, scheduling the use of a classroom.

Set of activities $S = \{a_1, \dots, a_n\}$.

a_i needs resource during period $[s_i, f_i)$, which is a half-open interval, where s_i = start time and f_i = finish time.

Goal

Select the largest possible set of nonoverlapping (***mutually compatible***) activities.

Note

Could have many other objectives:

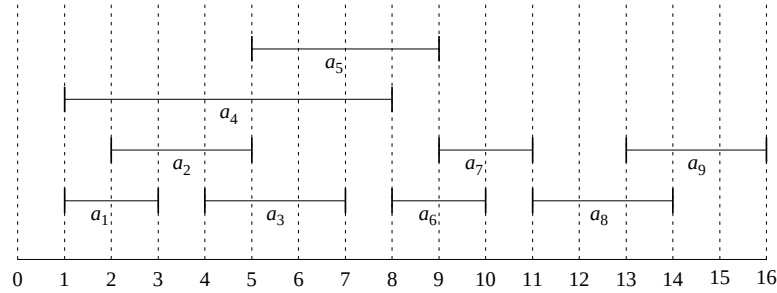
- Schedule room for longest time.
- Maximize income rental fees.

Assume that activities are sorted by finish time: $f_1 \leq f_2 \leq f_3 \leq \dots \leq f_{n-1} \leq f_n$.

Example

S sorted by finish time: [Leave on board]

i	1	2	3	4	5	6	7	8	9
s_i	1	2	4	1	5	8	9	11	13
f_i	3	5	7	8	9	10	11	14	16

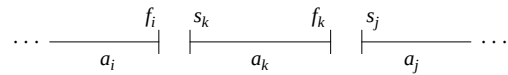


Maximum-size mutually compatible set: $\{a_1, a_3, a_6, a_8\}$.

Not unique: also $\{a_2, a_5, a_7, a_9\}$.

Optimal substructure of activity selection

$$\begin{aligned}
 S_{ij} &= \{a_k \in S : f_i \leq s_k < f_k \leq s_j\} && \text{[Leave on board]} \\
 &= \text{activities that start after } a_i \text{ finishes and finish before } a_j \text{ starts.}
 \end{aligned}$$



Activities in S_{ij} are compatible with

- all activities that finish by f_i , and
- all activities that start no earlier than s_j .

Let A_{ij} be a maximum-size set of mutually compatible activities in S_{ij} .

Let $a_k \in A_{ij}$ be some activity in A_{ij} . Then we have two subproblems:

- Find mutually compatible activities in S_{ik} (activities that start after a_i finishes and that finish before a_k starts).
- Find mutually compatible activities in S_{kj} (activities that start after a_k finishes and that finish before a_j starts).

Let

$$A_{ik} = A_{ij} \cap S_{ik} = \text{activities in } A_{ij} \text{ that finish before } a_k \text{ starts,}$$

$$A_{kj} = A_{ij} \cap S_{kj} = \text{activities in } A_{ij} \text{ that start after } a_k \text{ finishes.}$$

$$\text{Then } A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$$

$$\Rightarrow |A_{ij}| = |A_{ik}| + |A_{kj}| + 1.$$

Claim

Optimal solution A_{ij} must include optimal solutions for the two subproblems for S_{ik} and S_{kj} .

Proof Use the usual cut-and-paste argument. Will show the claim for S_{kj} ; proof for S_{ik} is symmetric.

Suppose we could find a set A'_{kj} of mutually compatible activities in S_{kj} , where $|A'_{kj}| > |A_{kj}|$. Then use A'_{kj} instead of A_{kj} when solving the subproblem for S_{ij} . Size of resulting set of mutually compatible activities would be $|A_{ik}| + |A'_{kj}| + 1 > |A_{ik}| + |A_{kj}| + 1 = |A|$. Contradicts assumption that A_{ij} is optimal. ■ (claim)

One recursive solution

Since optimal solution A_{ij} must include optimal solutions to the subproblems for S_{ik} and S_{kj} , could solve by dynamic programming.

Let $c[i, j]$ = size of optimal solution for S_{ij} . Then

$$c[i, j] = c[i, k] + c[k, j] + 1.$$

But we don't know which activity a_k to choose, so we have to try them all:

$$c[i, j] = \begin{cases} 0 & \text{if } S_{ij} = \emptyset, \\ \max_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & \text{if } S_{ij} \neq \emptyset. \end{cases}$$

Could then develop a recursive algorithm and memoize it. Or could develop a bottom-up algorithm and fill in table entries.

Instead, we will look at a greedy approach.

Making the greedy choice

Choose an activity to add to optimal solution *before* solving subproblems. For activity-selection problem, we can get away with considering only the greedy choice: the activity that leaves the resource available for as many other activities as possible.

Question: Which activity leaves the resource available for the most other activities?
Answer: The first activity to finish. (If more than one activity has earliest finish time, can choose any such activity.)

Since activities are sorted by finish time, just choose activity a_1 .

That leaves only one subproblem to solve: finding a maximum size set of mutually compatible activities that start after a_1 finishes. (Don't have to worry about activities that finish before a_1 starts, because $s_1 < f_1$ and no activity a_i has finish time $f_i < f_1 \Rightarrow$ no activity a_i has $f_i \leq s_1$.)

Since have only subproblem to solve, simplify notation:

$$S_k = \{a_i \in S : s_i \geq f_k\} = \text{activities that start after } a_k \text{ finishes}.$$

Making greedy choice of $a_1 \Rightarrow S_1$ remains as only subproblem to solve. [Slight abuse of notation: referring to S_k not only as a set of activities but as a subproblem consisting of these activities.]

By optimal substructure, if a_1 is in an optimal solution, then an optimal solution to the original problem consists of a_1 plus all activities in an optimal solution to S_1 .

But need to prove that a_1 is always part of some optimal solution.

Theorem

If S_k is nonempty and a_m has the earliest finish time in S_k , then a_m is included in some optimal solution.

Proof Let A_k be an optimal solution to S_k , and let a_j have the earliest finish time of any activity in A_k . If $a_j = a_m$, done. Otherwise, let $A'_k = A_k - \{a_j\} \cup \{a_m\}$ be A_k but with a_m substituted for a_j .

Claim

Activities in A'_k are disjoint.

Proof Activities in A_k are disjoint, a_j is first activity in A_k to finish, and $f_m \leq f_j$. ■ (claim)

Since $|A'_k| = |A_k|$, conclude that A'_k is an optimal solution to S_k , and it includes a_m . ■ (theorem)

So, don't need full power of dynamic programming. Don't need to work bottom-up.

Instead, can just repeatedly choose the activity that finishes first, keep only the activities that are compatible with that one, and repeat until no activities remain.

Can work top-down: make a choice, then solve a subproblem. Don't have to solve subproblems before making a choice.

Recursive greedy algorithm

Start and finish times are represented by arrays s and f , where f is assumed to be already sorted in monotonically increasing order.

To start, add fictitious activity a_0 with $f_0 = 0$, so that $S_0 = S$, the entire set of activities.

Procedure REC-ACTIVITY-SELECTOR takes as parameters the arrays s and f , index k of current subproblem, and number n of activities in the original problem.

REC-ACTIVITY-SELECTOR(s, f, k, n)

$m = k + 1$

while $m \leq n$ and $s[m] < f[k]$ // find the first activity in S_k to finish

$m = m + 1$

if $m \leq n$

return $\{a_m\} \cup \text{REC-ACTIVITY-SELECTOR}(s, f, m, n)$

else return $\emptyset$

Initial call

REC-ACTIVITY-SELECTOR($s, f, 0, n$).

Idea

The **while** loop checks $a_{k+1}, a_{k+2}, \dots, a_n$ until it finds an activity a_m that is compatible with a_k (need $s_m \geq f_k$).

- If the loop terminates because a_m is found ($m \leq n$), then recursively solve S_m , and return this solution, along with a_m .
- If the loop never finds a compatible a_m ($m > n$), then just return empty set.

Go through example given earlier. Should get $\{a_1, a_3, a_6, a_8\}$.

Time

$\Theta(n)$ —each activity examined exactly once, assuming that activities are already sorted by finish times.

Iterative greedy algorithm

Can convert the recursive algorithm to an iterative one. It's already almost tail recursive.

GREEDY-ACTIVITY-SELECTOR(s, f)

```

 $n = s.length$ 
 $A = \{a_1\}$ 
 $k = 1$ 
for  $m = 2$  to  $n$ 
    if  $s[m] \geq f[k]$ 
         $A = A \cup \{a_m\}$ 
         $k = m$ 
return  $A$ 

```

Go through example given earlier. Should again get $\{a_1, a_3, a_6, a_8\}$.

Time

$\Theta(n)$, if activities are already sorted by finish times.

For both the recursive and iterative algorithms, add $O(n \lg n)$ time if activities need to be sorted.

Greedy strategy

The choice that seems best at the moment is the one we go with.

What did we do for activity selection?

1. Determine the optimal substructure.
2. Develop a recursive solution.
3. Show that if we make the greedy choice, only one subproblem remains.

4. Prove that it's always safe to make the greedy choice.
5. Develop a recursive greedy algorithm.
6. Convert it to an iterative algorithm.

At first, it looked like dynamic programming. In the activity-selection problem, we started out by defining subproblems S_{ij} , where both i and j varied. But then found that making the greedy choice allowed us to restrict the subproblems to be of the form S_k .

Could instead have gone straight for the greedy approach: in our first crack at defining subproblems, use the S_k form. Could then have proven that the greedy choice a_m (the first activity to finish), combined with optimal solution to the remaining compatible activities S_m , gives an optimal solution to S_k .

Typically, we streamline these steps:

1. Cast the optimization problem as one in which we make a choice and are left with one subproblem to solve.
2. Prove that there's always an optimal solution that makes the greedy choice, so that the greedy choice is always safe.
3. Demonstrate optimal substructure by showing that, having made the greedy choice, combining an optimal solution to the remaining subproblem with the greedy choice gives an optimal solution to the original problem.

No general way to tell whether a greedy algorithm is optimal, but two key ingredients are

1. greedy-choice property and
2. optimal substructure.

Greedy-choice property

Can assemble a globally optimal solution by making locally optimal (greedy) choices.

Dynamic programming

- Make a choice at each step.
- Choice depends on knowing optimal solutions to subproblems. Solve subproblems *first*.
- Solve *bottom-up*.

Greedy

- Make a choice at each step.
- Make the choice *before* solving the subproblems.
- Solve *top-down*.

Typically show the greedy-choice property by what we did for activity selection:

- Look at an optimal solution.

- If it includes the greedy choice, done.
- Otherwise, modify the optimal solution to include the greedy choice, yielding another solution that's just as good.

Can get efficiency gains from greedy-choice property.

- Preprocess input to put it into greedy order.
- Or, if dynamic data, use a priority queue.

Optimal substructure

Just show that optimal solution to subproblem and greedy choice $\Rightarrow$ optimal solution to problem.

Greedy vs. dynamic programming

The knapsack problem is a good example of the difference.

0-1 knapsack problem

- n items.
- Item i is worth v_i , weighs w_i pounds.
- Find a most valuable subset of items with total weight $\leq W$.
- Have to either take an item or not take it—can't take part of it.

Fractional knapsack problem

Like the 0-1 knapsack problem, but can take fraction of an item.

Both have optimal substructure.

But the fractional knapsack problem has the greedy-choice property, and the 0-1 knapsack problem does not.

To solve the fractional problem, rank items by value/weight: v_i/w_i . Let $v_i/w_i \geq v_{i+1}/w_{i+1}$ for all i . Take items in decreasing order of value/weight. Will take all of the items with the greatest value/weight, and possibly a fraction of the next item.

FRACTIONAL-KNAPSACK(v, w, W)

$load = 0$

$i = 1$

while $load < W$ and $i \leq n$

if $w_i \leq W - load$

 take all of item i

else take $(W - load)/w_i$ of item i

 add what was taken to $load$

$i = i + 1$

Time: $O(n \lg n)$ to sort, $O(n)$ thereafter.

Greedy doesn't work for the 0-1 knapsack problem. Might get empty space, which lowers the average value per pound of the items taken.

i	1	2	3
v_i	60	100	120
w_i	10	20	30
v_i/w_i	6	5	4

$W = 50$.

Greedy solution:

- Take items 1 and 2.
- value = 160, weight = 30.

Have 20 pounds of capacity left over.

Optimal solution:

- Take items 2 and 3.
- value = 220, weight = 50.

No leftover capacity.